

"""Pandas Dataframe"""

```
from google.colab import files
uploaded = files.upload()
```

Question.1

```
import pandas as pd
# Create a dictionary with the specified columns and some sample data
data = {
    'Product': ['Laptop', 'Mouse', 'Keyboard', 'Monitor', 'Webcam'],
    'Region': ['North', 'South', 'East', 'West', 'North'],
    'Sales': [1200, 25, 75, 300, 50],
    'Quantity': [10, 50, 30, 5, 20]
}
# Create the DataFrame
df = pd.DataFrame(data)
df.head()
```

```
...
```

	Product	Region	Sales	Quantity
0	Laptop	North	1200	10
1	Mouse	South	25	50
2	Keyboard	East	75	30
3	Monitor	West	300	5
4	Webcam	North	50	20

```
import pandas as pd
# Load the Retail_Dataset.csv file for questions 2-10
df_retail = pd.read_csv('Retail_dataset.csv')
df_retail.tail()
# Add 5 rows of sample data to df_retail
# Create a dictionary for 5 new sample rows, aligning with df_retail's
columns
new_sample_data = {
    'Order_ID': [1091, 1092, 1093, 1094, 1095],
    'Customer': ['David', 'Eve', 'Frank', 'Grace', 'Henry'],
    'Product': ['Headphones', 'Speaker', 'Webcam', 'Microphone',
'Charger'],
    'Region': ['East', 'West', 'North', 'South', 'East'],
    'Sales': [150.00, 200.00, 75.00, 120.00, 40.00],
    'Quantity': [3, 2, 1, 2, 5]
}
# Create a DataFrame from the new sample data
df_new_rows = pd.DataFrame(new_sample_data)
# Append the new rows to the existing df_retail DataFrame
```

```
df_retail = pd.concat([df_retail, df_new_rows], ignore_index=True)
```

Question.2 (Data Inspection)

Display the head to show original data and tail to show new appended data

```
df_retail.head()
```

```
df_retail.tail()
```

```
df_retail.info()
```

```
df_retail.shape
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 95 entries, 0 to 94
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   Order_ID    95 non-null    int64
 1   Customer    95 non-null    object
 2   Product     95 non-null    object
 3   Region      95 non-null    object
 4   Sales       95 non-null    float64
 5   Quantity    95 non-null    int64
dtypes: float64(1), int64(2), object(3)
memory usage: 4.6+ KB
(95, 6)
```

Question.3

Select and display only 'Product', 'Region', and 'Sales' columns

```
display(df_retail[['Product', 'Region', 'Sales']].head())
```

	Product	Region	Sales
0	Laptop	West	1056.0
1	Tablet	South	904.0
2	Laptop	South	1597.0
3	Keyboard	North	777.0
4	Phone	South	2028.0

Question.4 (Indexing using .loc)

Display the row with index 10 using .iloc

```
display(df_retail.iloc[10])
# Display the Sales value of the row with index 20
display(df_retail.iloc[20]['Sales'])
```

10

Order_ID	1011
Customer	Raj
Product	Monitor
Region	North
Sales	1693.0
Quantity	6

```
dtype: object
np.float64(1021.0)
```

```
# Question.5 (Indexing using .iloc)
# Display the first 6 rows and first 4 columns using .iloc
display(df_retail.iloc[:6, :4])
# Display the Sales value from the 3rd row using .iloc
sales_third_row = df_retail.iloc[2]['Sales']
print(f"Sales value from the 3rd row: {sales_third_row}")
```

	Order_ID	Customer	Product	Region
0	1001	Sneha	Laptop	West
1	1002	Neha	Tablet	South
2	1003	Meera	Laptop	South
3	1004	Rohan	Keyboard	North
4	1005	Priya	Phone	South
5	1006	Rahul	Tablet	West

Sales value from the 3rd row: 1597.0

```
# Question.6 (Row Slicing)
```

```
# Display the first 10 rows of the dataset using .iloc
display(df_retail.iloc[:10])
# Display rows 15 to 25 (inclusive) using .iloc
display(df_retail.iloc[15:26])
```

	Order_ID	Customer	Product	Region	Sales	Quantity
0	1001	Sneha	Laptop	West	1056.0	2
1	1002	Neha	Tablet	South	904.0	7
2	1003	Meera	Laptop	South	1597.0	5
3	1004	Rohan	Keyboard	North	777.0	8
4	1005	Priya	Phone	South	2028.0	4
5	1006	Rahul	Tablet	West	638.0	2
6	1007	Amit	Monitor	South	1004.0	7
7	1008	Sneha	Monitor	East	1269.0	8
8	1009	Priya	Phone	North	827.0	8
9	1010	Amit	Phone	West	1998.0	4

	Order_ID	Customer	Product	Region	Sales	Quantity
15	1016	Karan	Monitor	West	796.0	3
16	1017	Neha	Laptop	East	2015.0	2
17	1018	Anita	Phone	North	1637.0	3
18	1019	Amit	Phone	East	748.0	7
19	1020	Neha	Laptop	West	1503.0	6
20	1021	Priya	Laptop	North	1021.0	4
21	1022	Raj	Tablet	South	886.0	7
22	1023	Rahul	Laptop	South	2267.0	3
23	1024	Karan	Monitor	East	814.0	7
24	1025	Meera	Monitor	West	1926.0	6
25	1026	Anita	Tablet	East	839.0	5

Question.7 (Filtering Data)

```
# Filter and display rows where Region = "East"
display(df_retail[df_retail['Region'] == 'East'].head())
# Filter and display rows where Sales > 1500
display(df_retail[df_retail['Sales'] > 1500].head())
```

	Order_ID	Customer	Product	Region	Sales	Quantity
7	1008	Sneha	Monitor	East	1269.0	8
11	1012	Rahul	Laptop	East	1022.0	5
16	1017	Neha	Laptop	East	2015.0	2
18	1019	Amit	Phone	East	748.0	7
23	1024	Karan	Monitor	East	814.0	7

	Order_ID	Customer	Product	Region	Sales	Quantity
2	1003	Meera	Laptop	South	1597.0	5
4	1005	Priya	Phone	South	2028.0	4
9	1010	Amit	Phone	West	1998.0	4
10	1011	Raj	Monitor	North	1693.0	6
13	1014	Rohan	Phone	South	2078.0	3

```
# Question.8 (Multiple Condition Filtering)
# Filter and display rows where Region = 'West' AND Sales > 1200
display(df_retail[(df_retail['Region'] == 'West') & (df_retail['Sales'] > 1200)].head())
```

	Order_ID	Customer	Product	Region	Sales	Quantity
9	1010	Amit	Phone	West	1998.0	4
19	1020	Neha	Laptop	West	1503.0	6
24	1025	Meera	Monitor	West	1926.0	6
29	1030	Rohan	Monitor	West	1932.0	5
42	1043	Meera	Keyboard	West	1515.0	6

```
# Question.9 (Descriptive Statistics)
# Generate summary statistics for df_retail
```

```

summary_stats = df_retail.describe()
display(summary_stats)
# Identify Mean sales value
mean_sales = summary_stats.loc['mean', 'Sales']
print(f"Mean Sales Value: {mean_sales:.2f}")
# Identify Maximum quantity sold
max_quantity = summary_stats.loc['max', 'Quantity']
print(f"Maximum Quantity Sold: {max_quantity:.0f}")
# Identify Minimum sales value
min_sales = summary_stats.loc['min', 'Sales']
print(f"Minimum Sales Value: {min_sales:.2f}")

```

	Order_ID	Sales	Quantity
count	95.000000	95.000000	95.000000
mean	1048.000000	1339.000000	4.589474
std	27.568098	593.064598	2.195231
min	1001.000000	40.000000	1.000000
25%	1024.500000	868.500000	2.500000
50%	1048.000000	1349.000000	5.000000
75%	1071.500000	1833.000000	6.500000
max	1095.000000	2455.000000	8.000000

Mean Sales Value: 1339.00

Maximum Quantity Sold: 8

Minimum Sales Value: 40.00

```

# Question.10 (Aggregation Analysis)
# Calculate total sales
total_sales = df_retail['Sales'].sum()
print(f"Total Sales: {total_sales:.2f}")
# Calculate average sales
average_sales = df_retail['Sales'].mean()
print(f"Average Sales: {average_sales:.2f}")
# Calculate maximum sales
max_sales = df_retail['Sales'].max()
print(f"Maximum Sales: {max_sales:.2f}")
# Calculate minimum sales
min_sales = df_retail['Sales'].min()
print(f"Minimum Sales: {min_sales:.2f}")

```

Total Sales: 127205.00
Average Sales: 1339.00
Maximum Sales: 2455.00
Minimum Sales: 40.00